

Local Competition Modules

A local Screeps competition module

[docs/competitions/README.md](#)

These modules turn the local Screeps private server into a replayable training lab. Each module is a short, resettable challenge against either engine NPC invaders or an autonomous bot colony, tied to one program skillset. Use these after the learner can connect to the local server and run their own code. Server setup lives in [docs/local-screeps.md](#).

Design Rules

- One module teaches one primary skill.
- The setup must be reproducible from local commands.
- The opponent pressure must be explicit: invader wave, `tooangel`, `zeswarm`, or timed benchmark.
- The win condition must be observable in-game.
- The reflection should ask for evidence from code, room state, and logs.
- The module should be short enough to replay in one lab block.

Tooling Available Locally

Tool	Best For	Command Surface
Sparring wave	Fast defense repetitions	<code>POST /local/api/sparring/wave</code>
<code>tooangel</code> bot	Persistent rival with attack behavior	<code>bots.spawn('tooangel', room, opts)</code>
<code>zeswarm</code> bot	Economy/race-style benchmark	<code>bots.spawn('zeswarm', room, opts)</code>
Admin CLI	World setup, map checks, cleanup	<code>docker exec -it autonate-screeps screeps-launcher cli</code>
Journal template	Debugging and reflection	<code>docs/journal/TEMPLATE.md</code>

Module Progression

#	Module	Skill Target	Pressure	Status
1	First Invasion Defense	Tower logic, hostile detection, survival triage	Sparring wave	Draft
2	Five-Wave Reliability	Regression testing under repeated pressure	Sparring wave loop	Draft
3	Scout Before Contact	Intel collection and room classification	Nearby bot colony	Draft
4	Economy Race	Spawn uptime, source assignment, RCL pace	<code>zeswarm</code> benchmark	Draft
5	Rival Colony Border	Expansion timing and threat response	<code>tooangel</code> nearby	Draft
6	League Scrimmage	Submission discipline and scoring	Bot or human opponent	Draft

Standard Run Flow

1. Start or restart the local server.
2. Confirm local mods are loaded.

3. Reset or choose the target room state.
4. Run the setup commands from the module.
5. Start the learner code.
6. Trigger the pressure.
7. Observe the win condition.
8. Capture a short journal entry.
9. Repeat with one code improvement.

Common Commands

Health check for sparring tools:

```
curl http://localhost:21025/local/api/sparring/health
```

Trigger one invader wave:

```
curl -X POST http://localhost:21025/local/api/sparring/wave \  
-H 'Content-Type: application/json' \  
-d '{"room": "W6N3"}'
```

Run repeated waves:

```
scripts/sparring-loop.sh W6N3 5 10
```

Open the server CLI:

```
docker exec -it autonate-sereeps sereeps-launcher cli
```

Spawn a bot rival:

```
bots.spawn('tooangel', 'W3N1', { cpu: 100, gcl: 1 })  
bots.spawn('zeswarm', 'W3N2', { cpu: 100, gcl: 1 })
```

Files

- `module-template.md` - copy this for new modules.
- `01-first-invasion-defense.md` - first combat automation checkpoint.
- `02-five-wave-reliability.md` - repeated pressure and regression testing.
- `03-scout-before-contact.md` - scouting and intel before action.
- `04-economy-race.md` - growth race against an economy bot.
- `05-rival-colony-border.md` - expansion under pressure from an attacker bot.